

The House That Words Built

Programming After the Blueprint

Watson Hartsoe

Working paper · 17 August 2026

Abstract

Natural-language programming is usually described as a change of interface: programmers can state intentions in ordinary language and let a model translate them into code. This paper argues that the deeper change is temporal and constitutional. Generative systems allow consequential making to begin before the thing to be made, the criteria by which it will be judged, and even parts of the maker's intention have been fully determined. The missing determinations do not vanish. They are supplied provisionally by models, tools, representational conventions, institutions, and later by the resistance of artifacts and worlds. I develop this argument through architecture because a house makes the distribution impossible to hide. A sentence about privacy becomes a plan, a plan becomes a structured building description, a description becomes machine and human work, and a roof that looks complete still waits for rain. Drawing on Austin, Winograd and Flores, Eastman, Stiny and Gips, Evans, Suchman, and especially Tim Ingold, I argue that the strongest model is not deferred formalization but *programming by correspondence*: description, artifact, test, material response, and revised intention remain in a loop in which no single representation has final authority. The paper includes a runnable companion, *Sentence House 01*, whose plan and constraint history change across staged linguistic interventions and whose memory persists across reloads. The demonstrator is not a construction document; it is an executable argument. Its central claim is that the house words can genuinely help build is not the output of a sentence. It is the accumulated history of who or what was allowed to continue the sentence, and of which worlds were allowed to answer back.

Keywords: natural-language programming; programming by correspondence; architecture; specification; generative AI; design computation; material practice; provenance

1. A house enters the paper

Begin with an instruction that is almost useless as a specification:

Build me a house that remembers.

What is a memory in a house? A photograph on a wall? A database of prior states? Wear in a floorboard? The accumulated repairs by which an inhabitant can read the building's history? A digital system that recognizes returning occupants? The sentence answers none of these questions. It says nothing about site, climate, structure, privacy, number of rooms, accessibility, cost, ownership, or who has the right to decide what the word *remembers* will mean.

Yet a generative system does not have to stop. It can make a first house.

That simple fact is more important than the debate over whether ordinary language is “really” a programming language. In conventional programming, an unresolved identifier or malformed expression is often a reason not to proceed. A generative model is organized to proceed. It can take a metaphor, a loose quality, an

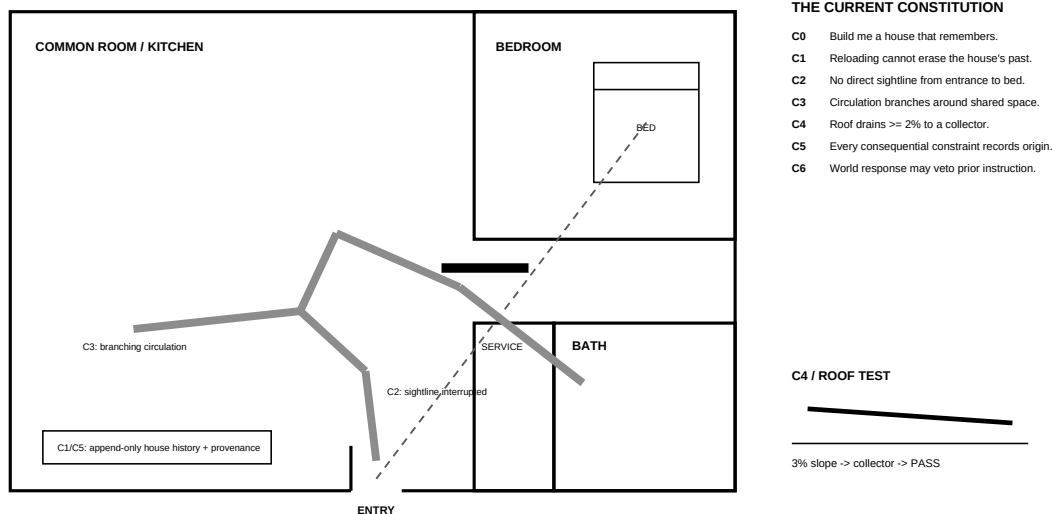
example, or an incomplete request and produce something determinate enough to inspect. The determinacy of the artifact can easily be mistaken for determinacy in the instruction. But the gap between them is where the most consequential action has occurred.

Something completed the sentence.

The companion artifact to this paper, *Sentence House 01*, makes that completion visible rather than hiding it. The initial plan contains a bedroom, common room, service spaces, an entrance, a roof condition, and a mechanism of memory. None of these follows deductively from the seed sentence. They are provisional continuations. The artifact then changes across a sequence of later instructions: history must survive reload; the entrance must not look directly onto the bed; circulation must branch rather than collapse into a single corridor; roof surfaces must drain; every consequential constraint must carry provenance; later worldly or inhabitant responses must append rather than erase what came before.

SENTENCE HOUSE 01 / CURRENT SETTLEMENT

research demonstrator - not construction documents



The house is not the output of a sentence. It is the history of who or what was allowed to continue the sentence.

Figure 1: Sentence House 01 at the current settlement of its constraint history. The figure is a research demonstrator, not a construction drawing. The important object is not the final plan but the sequence by which previously unstated distinctions became consequential.

The point of the exercise is not to pretend that a browser has built a habitable structure. It is to put a concrete object under the paper's claims. If "the house that words built" remains only a title, the argument can glide too easily between metaphor, software, drawing, fabrication, and actual building. The demonstrator imposes a discipline. At each stage we can ask: what did the words actually determine? What was supplied

by the system? What became visible only after an artifact existed? What required a test? What could only be learned after the artifact encountered a world? What remains outside the representation?

The stronger thesis emerges from these questions. Natural-language programming does not merely defer formalization until later in a conversation. In open-ended making, some determinations are not waiting in finished form to be recovered. They become available only through making. The program therefore cannot be understood as a complete object that exists either before execution or at the end of prompting. It is better understood as a history of continuations whose authority is repeatedly redistributed.

This is the argument I call *programming after the blueprint*.

2. The blueprint is the wrong temporal model

The blueprint organizes time in a familiar way. First a complete form is conceived. Then it is represented. Then workers or machines execute the representation. A modern software analogue is equally familiar:

specify → implement → execute.

Neither architecture nor programming has ever conformed perfectly to this sequence. Programs are debugged and refactored. Buildings change on site. Designers sketch to find out what they think. The value of the sequence is normative rather than descriptive: it locates intelligence before execution and treats later deviation as correction, error, or implementation detail.

Ingold's account of medieval building is useful because it attacks this temporal partition at the level of making itself. In his reading, masons could design while drawing and draw while designing; practical geometry unfolded on site through strings, templates, full-scale tracing, cutting, fitting, and adjustment (Ingold, 2013, 54–56). The design was not absent, but neither was it fully placed before the work. The intelligence of design remained inside the activity of building.

This changes the meaning of incompleteness. Under a blueprint model, an incomplete specification is a temporary deficit. More detail should eventually close the gap. Under an anticipatory model, incompleteness can be productive because certain decisions become meaningful only after earlier decisions have changed the field in which later decisions will be made. An opening in a wall changes the light. The light changes the furniture. The furniture changes circulation. The circulation changes what privacy means. There is no guarantee that all of these constraints were merely hidden in the original intention.

The distinction matters for generative programming. Suppose the first plan for *Sentence House 01* puts the bed in a direct sightline from the entrance. The user rejects the result and adds a constraint: no direct sightline from entry to bed. This looks like deferred formalization. A vague value, *private*, has become a more explicit geometric condition.

Now suppose the generated plan contains an unexpected window seat that changes how the user imagines the common room. The next instruction is not a clarification of something previously meant. It is a new desire caused by the artifact. The artifact has not merely exposed an omitted requirement; it has changed the project.

These two cases should not be collapsed:

artifact → failure → constraint

and

artifact → possibility → new intention.

The first is specification. The second is invention. A theory that treats every later instruction as the formalization of an earlier intention quietly reinstates the blueprint at a higher level: the final desire is imagined to have been latent from the beginning. Ingold's discussion of drawing provides a direct counterpressure. Drawing can proceed from a hunch rather than a finished inner picture, and its developing meaning need not remain under the maker's prior control (Ingold, 2013, 126–127). The work can teach the maker what there is to want.

This is why I no longer think *deferred formalization* is sufficient as the name for the phenomenon. It captures an important temporal shift, but it still makes formalization sound like the destination toward which every iteration is headed. The more radical possibility is that execution and representation participate in producing the object that will later be formalized. The program does not merely become explicit over time. Parts of it come into existence over time.

3. The first wall is an act of delegation

If an incomplete instruction nevertheless produces a determinate artifact, the missing determination has been supplied elsewhere.

This gives natural-language programming a political structure that the language of convenience tends to obscure. The user experiences freedom from specification, but the system experiences a need to continue. Every ambiguity that does not stop execution is therefore a decision allocated to another part of the arrangement.

Some allocations are harmless and desirable. “Make the spacing balanced” would be a poor high-level instruction if the system had to ask for every coordinate before acting. Delegation is not a pathology. It is how experts work together. Austin's speech-act theory already makes clear that words acquire force within procedures, roles, and circumstances that exceed the sentence itself (Austin, 1962). Winograd and Flores later emphasized language as a means of organizing action and commitment rather than as the passive transfer of representations (Winograd and Flores, 1986). The operative unit is an arrangement of participants and conventions.

Generative systems intensify this arrangement because they can fill in what a speaker leaves unsaid while preserving the appearance of a continuous conversational surface. Consider again *Sentence House 01*. The seed sentence did not specify a bedroom. If the system supplies one, it has imported a model of domestic life. It did not specify a rectangular plan. If the system uses one, it has imported another convention. It did not specify a front door, private property, a nuclear family, a climate, a construction method, or even that the remembered history should belong to the building rather than to its occupants. Each apparently reasonable continuation makes a world more concrete.

The most dangerous defaults are not necessarily the ones that fail. Failure becomes visible and attracts correction. A default that looks plausible can survive without ever acquiring a name. This creates an asymmetry in what an iterative “constraint history” remembers. Rejected assumptions leave receipts. Successful assumptions often disappear into the artifact.

The problem is therefore not only how to preserve constraints but how to preserve the *origin of commitments*. A mature system should be able to distinguish at least among determinations introduced by a user, inferred by a model, forced by a downstream formalism, required by an institution, discovered through a test, and retained because an artifact made a new possibility desirable. These sources are not interchangeable. They carry different claims to authority and different burdens of responsibility.

This is where the paper's earlier phrase *deferred authority* becomes useful, but it too must be sharpened. Authority is not simply deferred from user to model. It migrates repeatedly. A building code can overrule a designer. A structural calculation can overrule a visual preference. A material shortage can force substitution. A worker can discover that a nominally correct detail cannot be executed as drawn. An inhabitant can alter

the house after handover. Rain can overrule everyone.

What ordinary language increasingly provides is not sovereign control but a powerful routing surface through which these authorities are recruited, often invisibly. The sentence does not become matter. It enters a jurisdiction.

4. The house contains many languages

The phrase “natural-language programming” encourages a misleading picture in which one language replaces another. English appears at the top; code disappears beneath it. Architecture makes this picture untenable because a house crosses too many representational regimes.

Alberti’s distinction between lineaments and matter already establishes a consequential separation between the intellectual ordering of a building and its material construction (Alberti, 1988). Robin Evans later insisted that architectural drawing derives power from its difference from the thing represented, not from transparent identity with it (Evans, 1997). Translation is productive precisely because the mediums are not the same.

Computing created additional representational layers. Winograd’s SHRDLU accepted normal English dialogue and executed commands inside a restricted blocks world (Winograd, 1971). The important point was not that English had become a universal programming language. The system succeeded because a small world, a procedural architecture, a grammar, and a representation of objects and relations already existed beneath the dialogue. English was able to call into that world.

Eastman’s Building Description System made a different move. A building became a structured computational object consisting of parts, spaces, properties, and relations that could be stored and manipulated (Eastman et al., 1974). GLIDE then supplied a programming-language environment for operations on detailed representations of physical systems (Eastman and Henrion, 1977). Stiny and Gips showed another route: a grammar could take shapes rather than strings as primitives and generate further spatial forms through rule application (Stiny and Gips, 1972). APT and numerical-control languages brought formal statements still closer to physical consequence by defining machining operations that could be translated into machine-specific control (Smith, 1980).

These are not stages of one lineage. They are different answers to different technical problems. Their juxtaposition matters because it shows how many kinds of “language” can inhabit a single contemporary building process:

ordinary language → requirements → diagrams → geometry → building objects → code → machine
instructions → marks and forces.

At every arrow, information is selected, invented, discarded, or transformed. Some representations are propositional. Some are geometric. Some are executable. Some are indexical marks on a site. Some exist only as practiced judgement. Goodman was right to insist on “words or other symbols” rather than granting words a monopoly on worldmaking (Goodman, 1978). Wittgenstein’s image of language as an old city is equally suggestive: contemporary making does not take place in a newly built linguistic suburb alone, but in a layered city of inherited practices, notations, and forms (Wittgenstein, 2009, §18).

The present technical shift is therefore not English replacing code. It is ordinary language acquiring a privileged place from which heterogeneous representations can be requested, generated, revised, and connected. Natural language becomes infrastructural because it can route among older specialist languages while hiding much of the routing.

This hiding is both the attraction and the risk. Specialist representations expose their own limits. A type checker, a CAD constraint, a drawing convention, or a machine controller announces at least some of the

assumptions under which it operates. Conversational interaction can conceal those limits by providing a plausible continuation. The interface feels less formal because formal decisions have moved elsewhere.

The house that words built is thus always already a multilingual house.

5. The roof is where the blueprint breaks

A convincing sentence-to-house story must survive the roof.

The roof is useful because it makes visible the difference between a representation that appears complete and a thing that has entered the conditions under which it must work. A roof can be drawn beautifully, represented as valid geometry, fabricated to nominal dimensions, and still leak. The failure is not necessarily legible in the image. It appears in rain, wind, joints, workmanship, material aging, maintenance, and time.

Ingold's account of building repeatedly returns to this mismatch. Builders work in a world where plans encounter materials that resist, conditions that were not anticipated, and buildings that continue to decay and change after formal handover. He writes that completion is, at best, a legal fiction (Ingold, 2013, 47–48). The remark is not merely philosophical. It identifies a boundary for any theory that treats the final artifact as the end of the program.

The current version of *Sentence House 01* therefore contains a deliberately simple roof test. An earlier stage has a nominally complete flat roof. A later stage introduces the constraint that every roof surface must drain at no less than two percent to a collector. The change is not important as architecture; it is important as epistemology. Nothing in the appearance of the first plan forces the second rule. The rule appears because the house is exposed to a world in which water can contradict it.

This suggests a stronger loop than the familiar sequence of prompting:

describe → generate → inspect → correct.

For open-world artifacts, we need:

describe → generate → expose → observe → revise.

The term *expose* does real theoretical work. It asks what world is capable of producing the relevant failure. A screen can expose a visual hierarchy problem. A unit test can expose a function error. An adversarial environment may be required to expose a security defect. A load test may be required for a structure. A season may be required for moisture. Years of inhabitation may be required to reveal a domestic arrangement that is technically functional and socially intolerable.

Failure legibility is therefore not a property of an artifact alone. It is a relation among artifact, world, test, observer, and duration.

This matters for natural-language programming because cheap generation can distract from the expensive part. If a model can produce a thousand variants in seconds, the scarce resource may no longer be generation or even selection. It may be the construction of adequate worlds in which those variants can be contradicted. The limiting problem becomes not “can the model make another house?” but “what has this house been allowed to encounter?”

The roof also disciplines a popular metaphor. A sentence-to-wall pipeline often appears as:

sentence → model → toolpath → matter.

Ingold's critique of hylomorphism shows what this diagram suppresses. Following Simondon, he describes brick-making not as the imposition of abstract form upon passive matter but as the meeting of transforming

chains: the mould must be made, the clay prepared, forces opposed, materials brought into compatibility (Ingold, 2013, 25–26). A computational system that adds more arrows while preserving one-way authority from prior description to passive material remains hylomorphic in structure.

The stronger system must allow the arrow to return.

6. From feedback to correspondence

Here the central term changes.

Deferred formalization describes a process in which incomplete descriptions become more explicit over time. But if the world can alter not only the execution but the goal, then formalization is no longer the best name for the whole process. The more powerful concept is *correspondence*.

Ingold uses correspondence to describe making as an ongoing answering between attentive practitioners and active materials. The point is not that material has intentions in the human sense. It is that skilled action proceeds by attending to what the material does and allowing that response to alter what happens next. In his compressed formulation, “making is a process of correspondence” (Ingold, 2013, 31).

The phrase should not be imported into computing carelessly. A force sensor is not a mason. Closed-loop control is not automatically craftsmanship. Many feedback systems are organized around an invariant target. They sense deviation only in order to restore the pre-specified goal. That is structurally different from a process in which the response of the situation can revise the goal itself.

This distinction gives us a demanding test for generative systems. Consider two loops.

Control loop:

target → act → sense error → correct toward target.

Correspondence loop:

provisional intention → act → encounter → revise intention and action.

The first preserves the target. The second permits the target to learn.

This is why *Sentence House 01* records not only failures but amendments. A resident can append a mark. A later condition can veto an earlier rule. The stored history is not treated as an immutable blueprint but as a genealogy of commitments. The house “remembers” not by preserving a perfect original state but by refusing to erase the sequence through which its current state acquired authority.

The difference is significant for programming theory. Debugging often presumes that a correct behavior has been specified somewhere, even if imperfectly implemented. Test-driven development turns examples into increasingly explicit behavioral commitments. Programming by demonstration lets examples stand in for code. Generative interaction can include all of these, but it can also produce an artifact whose existence changes what the user wants. In that moment, execution is not only validating a program. It is participating in its authorship.

I propose *programming by correspondence* for this stronger class of interaction: programming in which partial descriptions produce provisional artifacts; artifacts are exposed to users, tests, tools, and worlds; those encounters can create new constraints or alter goals; and the history of accepted, rejected, and externally imposed commitments remains part of the operative state.

The concept is narrower than saying all programming is conversational and broader than saying prompting is iterative. Its defining feature is not repetition. It is the possibility of *worldly veto and goal revision inside the programming process*.

7. The program moves while it runs

Once correspondence becomes the unit, the familiar question “where is the program?” becomes difficult to answer.

It is not only in the prompt. The prompt contains too little. It is not only in the generated code. Code does not contain the provenance of the values it encodes. It is not only in the model. The model does not contain the site condition that later forces a change. It is not in the final artifact, because the artifact may continue to change through use and maintenance.

The program may not be a single thing anywhere.

Suchman made a related intervention against plans treated as exhaustive determinants of situated action: plans are resources within action rather than complete scripts for it (Suchman, 1987). Ingold makes the same pressure concrete in building. Medieval rules could guide masons without determining every move; they were resources integrated into practical knowledge (Ingold, 2013, 54–57). The fact that a rule exists does not establish that the rule contains the practice.

Natural-language programming magnifies this separation because it permits a small amount of explicit language to recruit an enormous amount of prior competence. A prompt such as “private” can work only because something downstream can choose a spatial interpretation. Ingold’s distinction between specification and telling is useful here. Stories can offer “guidance without specification” (Ingold, 2013, 110–111). They work not by carrying every instruction but by orienting a capable practitioner who knows how to continue.

A prompt may often function in exactly this way. It is not a compressed source program. It is guidance addressed to a system that already has ways of continuing.

This changes how we should think about prompt thickness. A long prompt is not necessarily more program-like. A short prompt is not necessarily less so. The relevant question is which continuations have been constrained, which remain delegated, and what happens when the delegation encounters a case the prior competence cannot settle.

The companion house makes one answer inspectable. Its operative state is an append-only ledger containing statements, origins, reasons, and tests. The ledger is not claimed to be the whole program. It is a deliberately thin representation of the part of the program that has become explicit through interaction. Its incompleteness is the point. A real building would require far more: structural systems, site information, climate data, code compliance, material specifications, procurement, labor, tolerances, sequencing, inspection, occupancy, maintenance, and histories no ledger could exhaust.

A useful programming environment should therefore resist the temptation to present the current formalization as total. It should be able to say: this constraint came from the user; this one was inferred by the model; this one is a default of the CAD system; this one appeared only after a test; this one is still an unexamined assumption. It should preserve not merely what is true of the current artifact but how that truth acquired standing.

Ingold’s observation that material properties are known as histories rather than as context-free attributes provides an illuminating analogy (Ingold, 2013, 30–31). “The roof drains” is thinner than “under this storm condition, version four ponded at the valley; the slope changed; the next test passed; wind-driven rain remains untested.” The second representation carries a reason for future action.

The program, in this view, is not a text. It is a living argument about what may happen next.

8. The house of language reverses, then doubles back

The title of this paper begins with an older philosophical house.

Heidegger's sentence "Language is the house of Being" places human beings within language rather than outside it as engineers of a symbolic machine (Heidegger, 1998, 239). Language is a dwelling and a condition of disclosure. The contemporary technical situation seems at first to reverse the metaphor. We can increasingly use language to specify, generate, coordinate, and sometimes fabricate parts of the environments in which we will physically dwell. Language appears to move from house to builder.

But a simple reversal is too triumphalist. It makes the same mistake as the one-way sentence-to-wall pipeline. The house that words built is never built by words alone. It depends on the infrastructures through which words acquire consequences, and it remains subject to actors and conditions that words do not control.

The better image doubles back.

We dwell in languages that give us categories such as room, privacy, owner, family, accessibility, property, bedroom, smart home, code, model, and prompt. We then use those categories to call technical systems into action. Those systems materialize provisional arrangements. We enter the arrangements and discover new meanings for the categories with which we began. The house changes the language from which the next house will be described.

This recursive relation is visible at smaller scales throughout the genealogy. Alexander's architectural pattern language migrated into software design patterns, returning concepts from building to programming (Alexander et al., 1977). Architectural representation becomes computational structure in Eastman. Conversational computation becomes a design aspiration in Negroponte (Negroponte, 1970). Words, models, buildings, and practices repeatedly reorganize one another. The traffic is not one-way.

The phrase *house of language* can therefore name two simultaneous conditions. First, we remain creatures who inherit a world already partitioned by language. Second, language increasingly acts as infrastructure through which computational systems can partition the material world on our behalf. The first condition makes the second possible; the second feeds back into the first.

This is why the deepest problem is not whether prompts qualify as "real programming." That question remains trapped inside a professional boundary dispute. The harder question is what happens when ordinary language becomes a place where unresolved distinctions acquire operational consequences before they have become objects of explicit judgement.

Who is allowed to complete the sentence? Under what authority? With what memory? Against what test? And what is allowed to answer back?

9. Programming after the blueprint

The house now permits a more precise statement of the contribution.

Natural-language programming is distinctive not because iteration is new, because high-level languages are new, or because humans have never before used ordinary language to organize consequential action. All of those claims are historically weak. Craft practice, interactive programming, debugging, CAD, programming by demonstration, conversational systems, and ordinary organizational life already contain rich forms of iteration and delegation.

The contemporary novelty is a conjunction.

First, the input can have high semantic latitude. "Make it private" or "make the house remember" can initiate action without a domain-complete schema.

Second, artifact generation is cheap enough that partial intentions can be materialized computationally before they have been carefully formalized.

Third, the interval between description and artifact is compressed, so the artifact can quickly participate in the next act of description.

Fourth, the artifacts can call into mature executable infrastructures: programming languages, APIs, CAD systems, databases, simulators, machine-control systems, and institutional workflows.

Fifth, a large share of the determination required to cross these infrastructures can remain hidden behind a conversational surface.

Together these conditions alter the temporal organization of programming. Specification no longer needs to stand wholly before execution. But the resulting freedom is not the disappearance of formality. It is a redistribution of where formal commitments are made and who is entitled to make them.

The resulting design problem is therefore constitutional. Systems need mechanisms for at least four things:

1. **Provenance of commitments.** A consequential rule should say whether it came from the user, model inference, tool semantics, institutional requirement, test, or world response.
2. **Preservation of negative and positive history.** Systems should remember not only failures and exclusions but unexpected properties that changed the goal.
3. **Exposure to adequate worlds.** A visual artifact should not be treated as evidence for properties that require simulation, instrumentation, use, weather, or time.
4. **Reopenability.** A successful output should remain revisable when later encounters invalidate the conditions under which earlier commitments were accepted.

These requirements move beyond prompt engineering. They describe an environment for programming by correspondence.

They also give a more demanding meaning to the claim that the artifact is part of the specification. The artifact is not valuable merely because it makes hidden constraints visible to a human evaluator. It is valuable because it can enter worlds that language alone cannot exhaust. It can be run, inhabited, stressed, measured, misunderstood, repaired, and changed. The artifact becomes a site where the program meets what was not yet represented.

A program after the blueprint is therefore not specification-free. It is specification that remains answerable.

10. Conclusion: the house does not end at the period

The most misleading version of *The House That Words Built* would be a victory story about language conquering matter. It would begin with a sentence and end with a wall, celebrating the disappearance of the people, representations, machines, and materials in between.

The stronger claim is nearly the opposite.

We have built technical systems in which ordinary language can mobilize consequential machinery before its own meanings have settled. That capability lets people begin making from examples, metaphors, qualities, and partial intentions that conventional formal systems would require them to resolve earlier. It also means that unresolved meaning acquires operational consequences. The missing decisions are supplied somewhere. Artifacts can anchor those decisions before anyone notices that a choice was made. Tests can reveal some assumptions. Worlds can reveal others. Some defaults may remain invisible for years.

Architecture makes this distribution tangible. The sentence “private” can become a sightline rule. “Remember” can become an append-only history. A metaphor of flow can become a corridor, be rejected, and return as branching circulation. A roof can look finished until rain produces the condition under which drainage becomes a requirement. A resident can reopen a decision after legal completion. At every step, the house changes what the next sentence needs to say.

This is why the prompt is not the program. But the final constraint list is not the program either. Nor is the generated code. Nor is the model. In open-ended making, the operative program is distributed through a correspondence among descriptions, representations, tests, institutions, machines, materials, and inhabitants.

Its state includes not only what has been specified but which continuations have been authorized, which have been refused, and which worlds have been permitted to contradict the current settlement.

The companion house is deliberately modest. It does not prove that natural language can autonomously produce a safe, code-compliant, buildable dwelling. It proves a smaller and more useful point: even a toy house becomes intellectually different when its history of linguistic commitments is made executable and inspectable. The plan can change, the tests can change, the house can remember those changes, and the reason for each constraint can remain attached to it.

A real house would make the argument harder, not easier. It would introduce soil, gravity, moisture, labor, money, law, weather, maintenance, and people who refuse to live as the model predicted. That is precisely why the house belongs at the center of the theory rather than at its edge.

The house that words can genuinely help build is not the output of a sentence.

It is the history of who or what was allowed to continue the sentence.

And the difference between execution and making begins when the house is allowed to answer back.

A. Sentence House 01: executable research demonstrator

The paper is accompanied by `sentence-house.html`, a self-contained browser artifact, and `HOUSE_LEDGER.json`. The demonstrator is intentionally small enough that the source of each displayed constraint remains inspectable. It contains seven stages:

Stage	Origin	Consequential statement
C0	user	Build me a house that remembers.
C1	user after artifact	Reloading cannot erase the house's past.
C2	user after visible failure	No direct sightline from entrance to bed.
C3	user after rejection	Circulation branches rather than becoming one corridor.
C4	world test	Roof planes drain at no less than two percent.
C5	provenance rule	Every consequential constraint records its origin.
C6	constitution	World response may veto; inhabitant amendments append.

The artifact stores interaction marks in browser local storage so that its small history survives reload. It visualizes the privacy, circulation, and roof constraints and reports staged tests. These mechanisms are pedagogical. They do not constitute architectural verification, structural engineering, code analysis, or fabrication instructions. Their purpose is to instantiate the paper's central distinction between a final artifact and a history of authorized continuations.

References

- Leon Battista Alberti. *On the Art of Building in Ten Books*. Translated by Joseph Rykwert, Neil Leach, and Robert Tavernor. MIT Press, 1988.
- Christopher Alexander, Sara Ishikawa, Murray Silverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, 1977.
- J. L. Austin. *How to Do Things with Words*. Edited by J. O. Urmson. Clarendon Press, 1962.

- Charles Eastman, David Fisher, Gilles Lafue, Joseph Lividini, Douglas Stoker, and Christos Yessios. *An Outline of the Building Description System*. Research Report 50, Institute of Physical Planning, Carnegie-Mellon University, 1974.
- Charles M. Eastman and Max Henrion. “GLIDE: A Language for Design Information Systems.” In *Proceedings of the 4th Annual Conference on Computer Graphics and Interactive Techniques*, 24–33. ACM, 1977.
- Robin Evans. *Translations from Drawing to Building and Other Essays*. MIT Press, 1997.
- Nelson Goodman. *Ways of Worldmaking*. Hackett, 1978.
- Martin Heidegger. “Letter on ‘Humanism’.” In *Pathmarks*, edited by William McNeill. Cambridge University Press, 1998 [1946].
- Tim Ingold. *Making: Anthropology, Archaeology, Art and Architecture*. Routledge, 2013. doi:10.4324/9780203559055.
- Nicholas Negroponte. *The Architecture Machine: Toward a More Human Environment*. MIT Press, 1970.
- Donald A. Schön. *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, 1983.
- Bradford M. Smith. *Guidelines for Exchangeable APT Data Packages: APT Part Programmer’s Manual*. NBSIR 80-2073.2, National Bureau of Standards, 1980.
- George Stiny and James Gips. “Shape Grammars and the Generative Specification of Painting and Sculpture.” In *Information Processing 71*, 1460–1465. North-Holland, 1972.
- Lucy A. Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.
- Terry Winograd. *Procedures as a Representation for Data in a Computer Program for Understanding Natural Language*. MIT AI Technical Report 235, Massachusetts Institute of Technology, 1971.
- Terry Winograd and Fernando Flores. *Understanding Computers and Cognition: A New Foundation for Design*. Ablex, 1986.
- Ludwig Wittgenstein. *Philosophical Investigations*. 4th ed. Wiley-Blackwell, 2009 [1953].